

lambda 정리

이름 없는 함수를 짧게 만들어 STL 알고리즘에 전달하는 문법입니다.

사용처 (Where)

- sort의 정렬 기준 작성
- count_if/find_if 조건 작성
- 짧은 일회용 함수를 만들 때

방법 (How to Use)

- []는 캡처 목록
- ()는 매개변수
- {}는 함수 내용
- return으로 결과 반환

기본 정보

- 필요 헤더: 별도 헤더 없음
- 기본 선언: [](int a, int b) { return a < b; }

왜 써야 할까? (Why)

- 짧은 조건/기준을 가까운 곳에 작성 가능
- 별도 함수 이름을 만들 필요가 없음
- STL 알고리즘과 결합하면 표현력이 좋아짐

주요 함수

함수/문법	의미
[]	외부 변수 캡처 없음
[&]	외부 변수 참조 캡처
[=]	외부 변수 값 캡처
auto	복잡한 타입 간단히 받기

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {3, 1, 4, 2};
    sort(v.begin(), v.end(), [](int a, int b) {
        return a > b;
    });

    for (int x : v) cout << x << ' ';
}
```

lambda 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
호출 비용	작은 함수와 유사
sort와 함께	$O(n \log n)$ 비교마다 호출
count_if	$O(n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
표현력	의도를 타입으로 직접 표현	구조체/클래스를 직접 작성	배열/struct 수동 관리
코드 길이	짧고 표준적인 형태	필드와 생성자 작성 필요	초기화와 접근 실수 가능
유지보수	다른 STL과 자연스럽게 연동	설계가 늘어날수록 관리 필요	가독성이 떨어지기 쉬움
추천 상황	간단한 값 묶음/조건 전달	도메인 의미가 큰 객체	C API와 맞춰야 할 때

응용: 조건에 맞는 개수 세기

count_if에 lambda를 넘겨 특정 기준을 만족하는 원소만 셉니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};
    int limit = 3;

    int cnt = count_if(v.begin(), v.end(), [limit](int x) {
        return x > limit;
    });

    cout << cnt;
}
```

처음 배울 때 자주 하는 실수

- 캡처 방식을 무심코 [&]로 쓰면 값 변경 위험
- 비교 함수에서 <= 같은 조건을 쓰면 정렬 규칙이 깨질 수 있음
- 길어지면 별도 함수가 더 읽기 쉬움

비슷한 항목과 차이

- 일반 함수는 재사용에 좋음
- 함수 객체는 상태를 가진 비교자에 적합
- comparator는 정렬 기준 역할을 하는 함수/객체 전체를 말함

한 줄 정리: lambda은(는) 'sort의 정렬 기준 작성' 상황에서 먼저 떠올리면 좋은 STL입니다.